

Redes Neuronales sobre Grafos (GNN)

Técnicas de Inteligencia Artificial

Prof. Flavio Prieto

email: faprieto@unal.edu.co

INGENIERÍA MECATRONICA

Facultad de Ingeniería

Universidad Nacional de Colombia Sede Bogotá



7 de mayo de 2026

Redes Neuronales sobre Grafos (GNN)

Las GNN (Graph Neural Networks) o Redes Neuronales sobre Grafos^a son un tipo de modelo de aprendizaje profundo **diseñado para trabajar con datos que tienen estructura de grafo**, en lugar de estructuras tradicionales como vectores o imágenes.

^aMaterial tomado de: [A Gentle Introduction to Graph Neural Networks](#). y [Understanding Convolutions on Graphs](#).

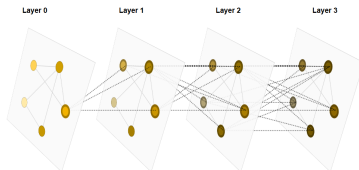


Figura tomada de: [Internet](#).

Datos en Forma de Grafos

- ▶ En el mundo real, muchos sistemas se describen mediante **relaciones entre entidades**.
- ▶ Estas relaciones pueden representarse naturalmente como **grafos**.
- ▶ Un grafo modela:
 - ▶ **Objetos** (nodos)
 - ▶ **Conexiones** (aristas)
- ▶ Las **Graph Neural Networks (GNNs)** permiten procesar este tipo de datos.

- ▶ En muchos problemas del mundo real, los datos no están organizados en forma de grilla (como imágenes), sino como **entidades conectadas entre sí**:
 - ▶ Redes sociales (personas conectadas)
 - ▶ Redes de transporte
 - ▶ Moléculas (átomos enlazados)
 - ▶ Sistemas multi-robot o UAVs
 - ▶ Simulación de sistemas físicos
 - ▶ Detección de noticias falsas
 - ▶ Predicción de tráfico
- ▶ Los grafos permiten **modelar estructuras complejas donde las relaciones son tan importantes como los datos**.

Definición Formal de Grafo

- ▶ Un grafo representa entidades y sus relaciones de forma estructurada.
- ▶ Se define como:

$$G = (V, E, U)$$

- ▶ Donde:
 - ▶ V : conjunto de nodos (vértices)
 - ▶ E : conjunto de aristas (conexiones)
 - ▶ U : atributos globales del grafo

Atributos en un Grafo

- ▶ La información puede almacenarse a múltiples niveles dentro del grafo.
- ▶ Un grafo puede contener tres tipos de atributos:
 - ▶ **Nodos (V):**
 - ▶ Identidad del nodo
 - ▶ Número de vecinos
 - ▶ **Aristas (E):**
 - ▶ Peso de la conexión
 - ▶ Tipo de relación
 - ▶ **Globales (U):**
 - ▶ Número total de nodos
 - ▶ Propiedades globales (ej: camino más largo)

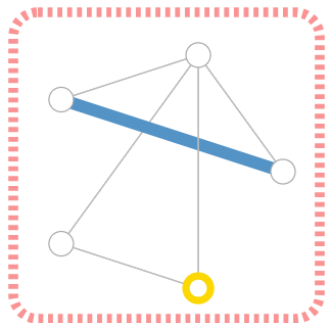
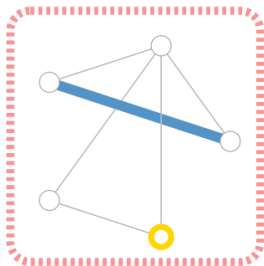


Figura tomada de: [Internet](#).

Representación mediante Embeddings

- ▶ La información en grafos puede representarse como:
 - ▶ Escalares
 - ▶ Vectores (embeddings): Permiten representar información estructural en espacios vectoriales.
- ▶ Tipos de representaciones:
 - ▶ Embedding de nodos: $h_v \in \mathbb{R}^d$
 - ▶ Embedding de aristas: $h_e \in \mathbb{R}^d$
 - ▶ Embedding global: $h_G \in \mathbb{R}^d$



Vertex (or node) embedding



Edge (or link) attributes and embedding



Global (or master node) embedding



Figura tomada de: [Internet](#).

Grafos Dirigidos y No Dirigidos

- ▶ Las aristas pueden ser:
 - ▶ **Dirigidas**
 - ▶ **No dirigidas**
- ▶ En grafos dirigidos:

$$e = (v_{src}, v_{dst})$$

- ▶ Flujo de información: $v_{src} \rightarrow v_{dst}$
- ▶ En grafos no dirigidos:
 - ▶ Flujo bidireccional

Undirected edge



Directed edge



Figura tomada de: [Internet](#).

- ▶ Un grafo no dirigido puede representarse como dos aristas dirigidas en sentidos opuestos.

Ejemplos de Grafos.

- ▶ Los grafos son una **representación general y poderosa** de datos.
- ▶ Permiten modelar estructuras más allá de ejemplos clásicos como redes sociales.
- ▶ Datos aparentemente distintos pueden representarse como grafos:
 - ▶ Imágenes
 - ▶ Texto

Imágenes como Grafos

- ▶ Una imagen se representa como: $X \in \mathbb{R}^{H \times W \times C}$
- ▶ Alternativamente, puede modelarse como un **grafo regular** donde cada píxel se conecta con sus vecinos:
 - ▶ Cada píxel $\rightarrow$ nodo
 - ▶ Vecindad espacial $\rightarrow$ aristas
- ▶ Cada nodo contiene: $h_v \in \mathbb{R}^3$ (RGB)

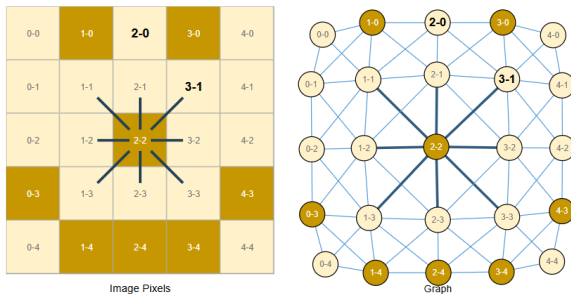


Figura tomada de: [Internet](#).

Conectividad en Imágenes

- ▶ En una grilla 2D, cada píxel interno tiene **8 vecinos**
- ▶ La conectividad puede representarse mediante la **matriz de adyacencia**: $A \in \mathbb{R}^{N \times N}$

donde: N es el número total de nodos (píxeles) del grafo, y

$$A_{ij} = \begin{cases} 1 & \text{si existe conexión entre } i \text{ y } j \\ 0 & \text{en otro caso} \end{cases}$$

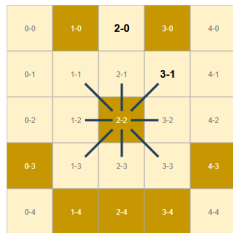
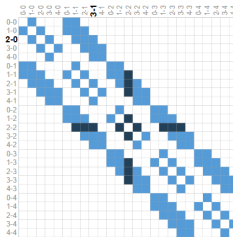


Image Pixels



Adjacency Matrix

Figura tomada de: [Internet](#).

Texto como Grafo

- ▶ El texto se digitaliza como una secuencia de tokens:

$$(w_1, w_2, \dots, w_T)$$

- ▶ Puede interpretarse como un **grafo secuencial dirigido**:
 - ▶ Cada token $\rightarrow$ nodo
 - ▶ Relación secuencial $\rightarrow$ aristas
- ▶ Conexiones:

$$w_t \rightarrow w_{t+1}$$

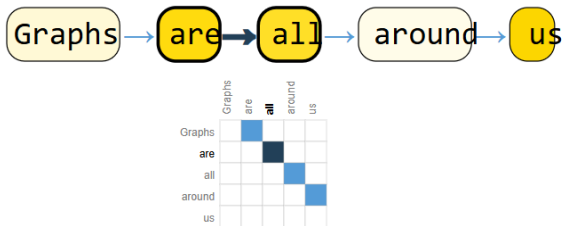


Figura tomada de: [Internet](#)

Estructura de la Matriz en Texto

- ▶ La matriz de adyacencia del texto es:
 - ▶ **Diagonal desplazada**
- ▶ Cada nodo se conecta únicamente con:
 - ▶ El anterior
 - ▶ El siguiente
- ▶ Representación: $A_{i,i+1} = 1$

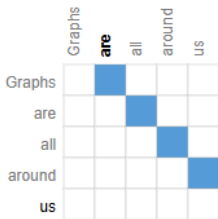


Figura tomada de: [Internet](#).

Grafos en el Mundo Real

- ▶ Los grafos son esenciales cuando la estructura no es uniforme ni regular.
- ▶ Normalmente no es así como se codifican los textos y las imágenes, ya que ellos presentan estructuras muy regulares.
 - ▶ Las imágenes tienen una estructura en banda en su matriz de adyacencia porque todos los nodos (píxeles) están conectados en una rejilla.
 - ▶ La matriz de adyacencia para el texto es simplemente una línea diagonal, porque cada palabra solo se conecta con la palabra anterior y con la siguiente.
- ▶ En datos reales:
 - ▶ La estructura es **irregular**
 - ▶ El número de vecinos es **variable**
- ▶ Estos datos son difíciles de representar sin grafos.

Moléculas como Grafos

- ▶ Los grafos permiten modelar la estructura química de manera natural.
- ▶ Las moléculas están formadas por: **átomos y enlaces químicos**
 - ▶ Átomos $\rightarrow$ nodos
 - ▶ Enlaces covalentes $\rightarrow$ aristas
- ▶ Propiedades:
 - ▶ Distancias variables
 - ▶ Tipos de enlace (simple, doble, etc.)

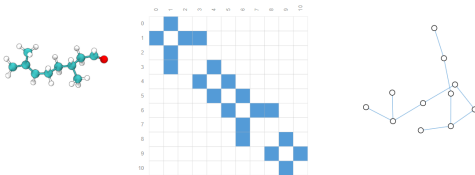


Figura tomada de: [Internet](#).

Redes Sociales como Grafos

- ▶ A diferencia de imágenes y texto, las redes sociales no presentan patrones regulares.
- ▶ Representación:
 - ▶ Personas → nodos
 - ▶ Relaciones → aristas
- ▶ Características:
 - ▶ Alta variabilidad en conexiones
 - ▶ Estructura no uniforme

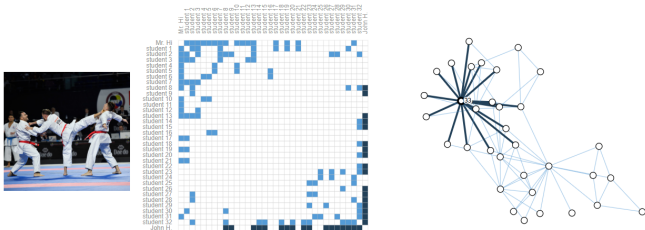


Figura tomada de: [Internet](#).

Redes de Citación

- ▶ Los grafos permiten integrar estructura y contenido semántico.
- ▶ En publicaciones científicas:
 - ▶ Artículos $\rightarrow$ nodos
 - ▶ Citaciones $\rightarrow$ aristas dirigidas
- ▶ Se puede enriquecer cada nodo con:
 - ▶ Embeddings del resumen

Variabilidad de los Grafos

- ▶ La heterogeneidad estructural es una característica clave de los grafos reales.
- ▶ Los grafos del mundo real presentan gran diversidad:
 - ▶ Número de nodos variable
 - ▶ Número de aristas variable
 - ▶ Diferente conectividad
- ▶ Ejemplos:
 - ▶ Grafos densos vs dispersos
 - ▶ Nodos con muchos o pocos vecinos

Tipos de Problemas en Grafos

- ▶ Los datos estructurados como grafos permiten resolver distintos tipos de tareas.
- ▶ Existen tres categorías principales:
 - ▶ **Nivel de grafo**
 - ▶ **Nivel de nodo**
 - ▶ **Nivel de arista**

Formalización General

- ▶ Sea un grafo: $G = (V, E)$
- ▶ Una GNN aprende una función: $f_{\theta}(G)$
- ▶ Dependiendo del problema:
 - ▶ Nivel grafo: $f_{\theta}(G) \rightarrow y$
 - ▶ Nivel nodo: $f_{\theta}(v_i) \rightarrow y_i$
 - ▶ Nivel arista: $f_{\theta}(e_{ij}) \rightarrow y_{ij}$

Tareas a Nivel de Grafo

- ▶ Objetivo:
 - ▶ Predecir una propiedad del grafo completo
- ▶ Ejemplo:
 - ▶ Moléculas → predecir propiedades químicas
- ▶ Formalmente:

$$y = f_{\theta}(G)$$

- ▶ Equivalente a clasificación de imágenes o análisis de sentimiento en texto.
- ▶ Toda la información estructural del grafo contribuye a una única predicción global.

Tipos de Problemas en Grafos

- ▶ Predicción de propiedades moleculares:
 - ▶ Olor
 - ▶ Afinidad con receptores biológicos
- ▶ Ejemplo:
 - ▶ **Entrada:** grafos
 - ▶ Salida: etiquetas para cada grafo (por ejemplo, “¿el grafo contiene dos anillos?”)

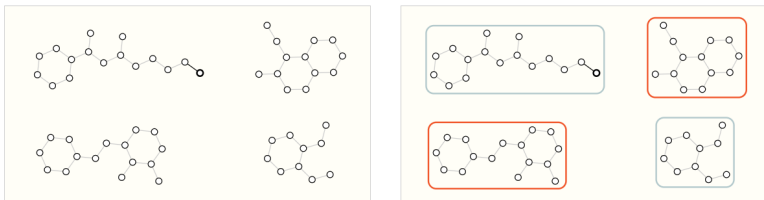


Figura tomada de: [Internet](#).

Tareas a Nivel de Nodo

- ▶ Objetivo:
 - ▶ Predecir una propiedad para cada nodo
- ▶ Formalmente:

$$y_i = f_{\theta}(v_i)$$

- ▶ Se busca identificar:
 - ▶ Rol
 - ▶ Clase
 - ▶ Estado del nodo

- ▶ Cada nodo se clasifica utilizando información de su vecindario.
- ▶ Las tareas de nodo implican predicciones locales dependientes del contexto.

Ejemplo: Karate Club

- ▶ Grafo social de individuos:
 - ▶ Nodos: personas
 - ▶ Aristas: interacciones
- ▶ Problema:
 - ▶ **Clasificar lealtad a dos grupos**
- ▶ Observación:
 - ▶ La distancia en el grafo influye en la predicción

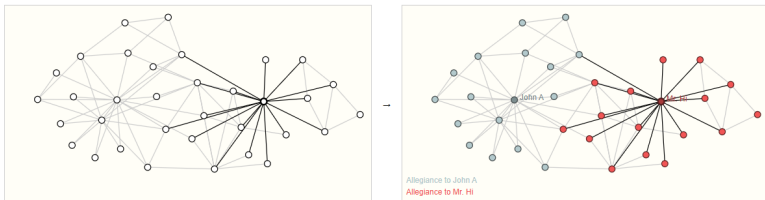


Figura tomada de: [Internet](#).

Tareas a Nivel de Arista

- ▶ Objetivo:
 - ▶ Predecir propiedades o existencia de aristas
- ▶ Formalmente:

$$y_{ij} = f_{\theta}(e_{ij})$$

- ▶ Se modelan relaciones entre pares de nodos.
- ▶ Las tareas de aristas permiten descubrir relaciones ocultas entre entidades.

Relación entre objetos en una escena

- ▶ La imagen original se segmenta en cinco entidades: cada uno de los luchadores, el árbitro, el público y la lona.



Figura tomada de: [Internet](#).

Tipos de Problemas en Grafos

Relación entre objetos en una escena

- ▶ Se muestra las relaciones entre estas entidades.

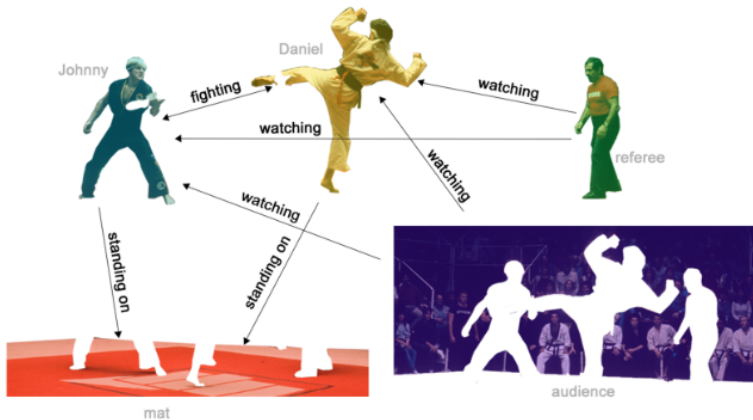


Figura tomada de: [Internet](#).

Relación entre objetos en una escena

- ▶ A la izquierda se tiene un grafo inicial construido a partir de la escena visual previa.
- ▶ A la derecha se muestra un posible etiquetado de las aristas de este grafo cuando algunas conexiones fueron podadas con base en la salida del modelo.

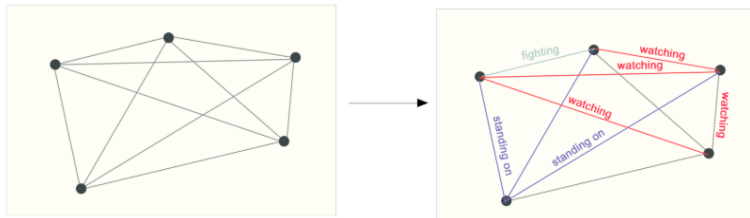


Figura tomada de: [Internet](#).

Unificación mediante GNNs

- ▶ Un mismo modelo GNN puede abordar:
 - ▶ Predicción global (grafo)
 - ▶ Predicción local (nodo)
 - ▶ Predicción relacional (arista)
- ▶ Diferencia principal:
 - ▶ Tipo de salida del modelo

Desafíos de Usar Grafos en Machine Learning

- ▶ El uso de grafos en redes neuronales plantea retos importantes.
- ▶ Problema principal:
 - ▶ ¿Cómo representar grafos para modelos de deep learning?
- ▶ Los modelos tradicionales esperan:
 - ▶ Datos en forma de **arreglos rectangulares**
- ▶ Los grafos no poseen una estructura regular, lo que dificulta su procesamiento directo.

Componentes de un Grafo

- ▶ Un grafo contiene hasta cuatro tipos de información:
 - ▶ Nodos
 - ▶ Aristas
 - ▶ Contexto global
 - ▶ Conectividad
- ▶ **Los tres primeros pueden representarse como tensores.**

Representación de Nodos y Atributos

- ▶ Se define una matriz de características de nodos:

$$N \in \mathbb{R}^{n_{\text{nodos}} \times d}$$

donde: n_{nodos} : número de nodos, y d : dimensión de características

- ▶ De forma similar:
 - ▶ Aristas: $E \in \mathbb{R}^{n_{\text{edges}} \times d_e}$. d_e : número de features por arista
 - ▶ Global: $u \in \mathbb{R}^{d_u}$
- ▶ Los atributos **pueden representarse como tensores**, incluso si su tamaño es variable.

El Problema de la Conectividad

- ▶ La conectividad del grafo es más difícil de representar.
- ▶ Opción común:
 - ▶ **Matriz de adyacencia**

$$A \in \mathbb{R}^{n_{\text{nodes}} \times n_{\text{nodes}}}$$

Limitaciones de la Matriz de Adyacencia

- ▶ Grafos reales pueden tener:
 - ▶ Millones de nodos
 - ▶ Conectividad variable
- ▶ Consecuencias:
 - ▶ Matrices muy grandes
 - ▶ Alta **esparsidad**
- ▶ Ineficiencia:

$$\text{Memoria} \sim O(n_{\text{nodes}}^2)$$

Problema de Invariancia a Permutaciones

- ▶ Un mismo grafo puede tener múltiples representaciones:
 - ▶ Diferentes ordenamientos de nodos
- ▶ Formalmente:

$$A' = PAP^T$$

- ▶ Donde P es una matriz de permutación
- ▶ **Problema:** Una red neuronal puede producir salidas distintas para representaciones equivalentes.
- ▶ Los modelos deben ser **invariantes a permutaciones**.

Crecimiento Combinatorio

- ▶ El número de representaciones posibles crece rápidamente:

$$n_{\text{nodes}}!$$

- ▶ Incluso grafos pequeños tienen muchas matrices equivalentes.

La ambigüedad en la representación dificulta el aprendizaje consistente.

Desafíos de Usar Grafos en Machine Learning

Ejemplo

- ▶ Todas las matrices de adyacencia representan el mismo grafo.

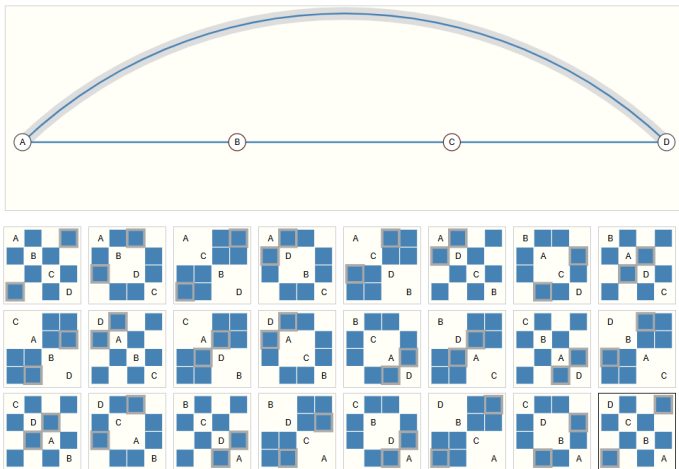


Figura tomada de: [Internet](#).

Listas de Adyacencia

- ▶ Son una forma elegante y eficiente en memoria de representar matrices dispersas.
- ▶ Representación:
$$e_k = (i, j)$$
- ▶ Cada entrada describe una arista entre nodos.
- ▶ Solo se almacenan conexiones existentes.

Ventajas de las Listas de Adyacencia

- ▶ Complejidad en memoria:

$$O(n_{\text{edges}})$$

- ▶ Beneficios:
 - ▶ Eficiencia en grafos dispersos
 - ▶ Evita almacenamiento innecesario
 - ▶ Escalable a grandes grafos

Ejemplo

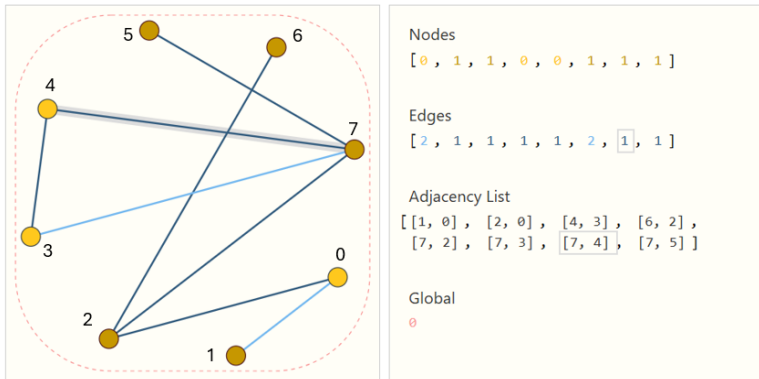


Figura tomada de: [Internet](#).

Representación Tensorial

- ▶ En la práctica **no se usan escalares, sino vectores**.
- ▶ Representaciones típicas:
 - ▶ Nodos: $[n_{\text{nodes}}, d]$
 - ▶ Aristas: $[n_{\text{edges}}, d_e]$
 - ▶ Global: $[d_u]$
- ▶ Permiten:
 - ▶ Aprendizaje profundo
 - ▶ Representaciones ricas

Extensión de Convoluciones a Grafos

- ▶ Las **Redes Neuronales Convolucionales (CNNs)** han demostrado ser **muy poderosas** para extraer características de imágenes.
- ▶ Las imágenes pueden interpretarse como **grafos con estructura regular en forma de rejilla**.
- ▶ En esta representación:
 - ▶ Los **píxeles** corresponden a los **nod**os.
 - ▶ Los valores **RGB** corresponden a las **características de los nodos**.

- ▶ Surge de manera natural la idea de **generalizar las convoluciones** a grafos arbitrarios.
- ▶ El objetivo es extender el mecanismo de las CNNs más allá de estructuras en rejilla.
- ▶ Sin embargo, existen **dificultades importantes**:
 - ▶ Las convoluciones tradicionales **no son invariantes al orden de los nodos**.
 - ▶ Estas dependen de las **posiciones absolutas** de los píxeles.
- ▶ Esto plantea un problema al trabajar con grafos generales.

Estructura de Vecindad

- En imágenes la vecindad es **regular y consistente**.

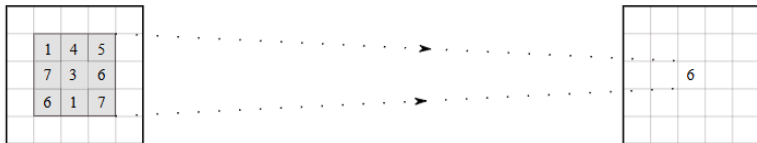


Figura tomada de: [Internet](#).

Extensión de Convoluciones a Grafos

- En grafos la estructura de vecindad **varía de nodo a nodo**.

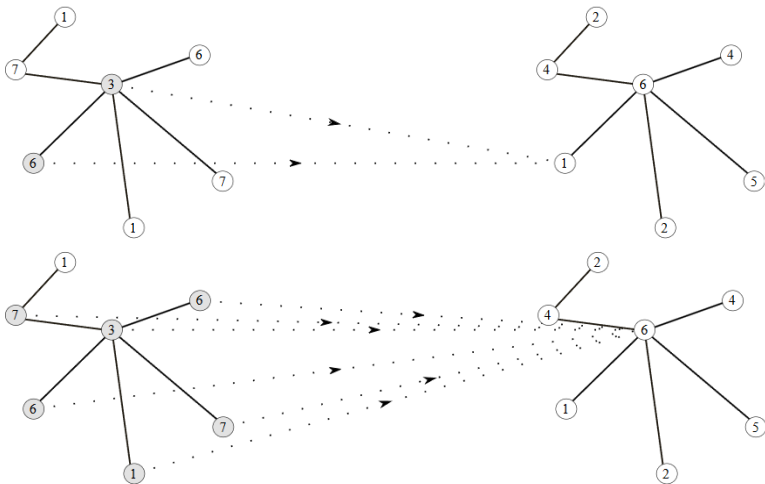


Figura tomada de: [Internet](#).

- ▶ La generalización de convoluciones a grafos **no es directa**.
- ▶ El principal reto es:
 - ▶ Manejar la **dependencia del orden** y la **variabilidad estructural**.
- ▶ Esto motiva el desarrollo de nuevos enfoques para grafos.

Filtros polinomiales sobre grafos

- ▶ Dado un grafo G , se fija un **orden arbitrario** de sus n nodos.
- ▶ Se define la **matriz de adyacencia**:
 - ▶ $\mathbf{A} \in \mathbb{R}^{n \times n}$
 - ▶ Cuyos valores de la matriz son **0** o **1**.
- ▶ La entrada $\mathbf{A}_{vu}$ indica la conexión entre los nodos v y u :
 - ▶ La entrada en la **fila correspondiente a v**
 - ▶ La **columna correspondiente a u**
- ▶ A partir de $\mathbf{A}$ se construye la **matriz diagonal de grados $\mathbf{D}$** .
- ▶ Sus elementos se definen como:

$$D_v = \sum_u A_{vu}$$

- ▶ Es decir, el grado de un nodo v es el **número de aristas incidentes** en v .

Extensión de Convoluciones a Grafos

► Matrices **A** y **D** para el grafo siguiente:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$D = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 5 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

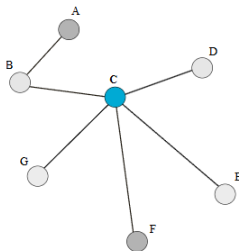
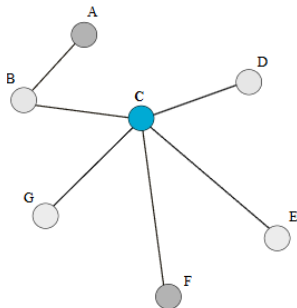


Figura tomada de: [Internet](#).

Laplaciano del Grafo

- ▶ El **Laplaciano del grafo** se define como: $L = D - A$
 - ▶ $L \in \mathbb{R}^{n \times n}$ es una matriz **cuadrada**.
 - ▶ Combina información de **grado** y **conectividad**.



Input Graph G

	A	B	C	D	E	F	G
A	1	-1					
B	-1	2	-1				
C		-1	5	-1	-1	-1	-1
D			-1	1			
E			-1		1		
F			-1			1	
G			-1				1

Laplacian L of G

Figura tomada de: [Internet](#).

Interpretación del Laplaciano

- ▶ El Laplaciano del grafo recibe su nombre por ser el **análogo discreto** del operador Laplaciano del cálculo.
- ▶ Permite trasladar ideas del **análisis continuo** al caso discreto de grafos.

Polinomios del Laplaciano

- ▶ Una vez definido el Laplaciano $\mathbf{L}$, se pueden construir **polinomios de la forma**:

$$p_{\mathbf{w}}(\mathbf{L}) = w_0 \mathbf{I}_n + w_1 \mathbf{L} + w_2 \mathbf{L}^2 + \cdots + w_d \mathbf{L}^d = \sum_{i=0}^d w_i \mathbf{L}^i$$

- ▶ Cada polinomio se representa mediante el vector de coeficientes:

$$\mathbf{w} = [w_0, \dots, w_d]$$

- ▶ **Idea central:**

- ▶ $p_{\mathbf{w}}(\mathbf{L})$ es una matriz $n \times n$, al igual que $\mathbf{L}$.
- ▶ Estos polinomios pueden interpretarse como **filtros análogos a los de CNNs**.
- ▶ Los coeficientes $\mathbf{w}$ corresponden a los **pesos del filtro**.

Vector de Características

- ▶ Se considera el caso donde cada nodo tiene una **característica escalar**: $x_v \in \mathbb{R}$
- ▶ Las características se agrupan en un vector: $\mathbf{x} \in \mathbb{R}^n$
- ▶ Las mismas ideas se extienden a características de mayor dimensión.
- ▶ **Ejemplo:** Se fija un orden de los nodos (indicado por las letras) y se agrupan todas las características de los nodos en un único vector $\mathbf{x}$

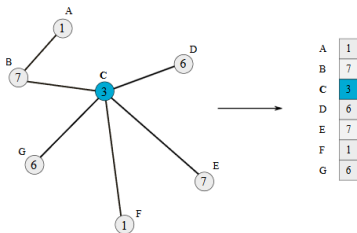


Figura tomada de: [Internet](#).

Convolución en Grafos

- ▶ La convolución con un filtro polinomial se define como:

$$\mathbf{x}' = p_{\mathbf{w}}(\mathbf{L})\mathbf{x}$$

- ▶ La señal en el grafo es transformada mediante el polinomio del Laplaciano.

Caso Simple: $w_0 = 1$

- ▶ Si $w_0 = 1$ y los demás coeficientes son 0:

$$x' = p_w(L)x = \sum_{i=0}^d w_i L^i x = w_0 I_n x = x$$

- ▶ La señal permanece **sin cambios**.

Caso $w_1 = 1$

- ▶ Si $w_1 = 1$ y los demás coeficientes son 0:

$$\mathbf{x}' = \mathbf{L}\mathbf{x}$$

- ▶ A nivel de nodo:

$$\begin{aligned}x'_v &= (\mathbf{L}\mathbf{x})_v = L_v x \\&= \sum_{u \in G} L_{vu} x_u \\&= \sum_{u \in G} (D_{vu} - A_{vu}) x_u \\&= D_v x_v - \sum_{u \in \mathcal{N}(v)} x_u\end{aligned}$$

- ▶ Cada nodo combina su valor con el de sus **vecinos inmediatos**.

Extensión de Convolutiones a Grafos

Partiendo de:

$$\sum_{u \in G} (D_{vu} - A_{vu})x_u$$

se separa la suma:

$$= \sum_{u \in G} D_{vu}x_u - \sum_{u \in G} A_{vu}x_u$$

Primer término: Matriz de grados

La matriz de grados D es diagonal:

$$D_{vu} = \begin{cases} D_v & \text{si } v = u \\ 0 & \text{si } v \neq u \end{cases}$$

Por tanto:

$$\sum_{u \in G} D_{vu}x_u = D_v x_v$$

ya que únicamente el término con $u = v$ es distinto de cero.

Segundo término: Matriz de adyacencia

La matriz de adyacencia cumple:

$$A_{vu} = \begin{cases} 1 & \text{si existe una arista entre } v \text{ y } u \\ 0 & \text{en otro caso} \end{cases}$$

Entonces:

$$\sum_{u \in G} A_{vu} x_u = \sum_{u \in \mathcal{N}(v)} x_u$$

ya que solo se seleccionan los nodos vecinos de v .

Influencia del Grado d del polinomio

- ▶ L^i solo conecta nodos a distancia menor o igual a i .

$$\text{dist}_G(v, u) > i \Rightarrow (L^i)_{vu} = 0$$

- ▶ Esto implica que, **al convolucionar $p_w(L)$ de grado d para obtener x'**

$$\begin{aligned} x'_v &= (p_w(L)x)_v = (p_w(L))_v x = \sum_{i=0}^d w_i L_v^i x \\ &= \sum_{i=0}^d w_i \sum_{u \in G} L_{vu}^i x_u = \sum_{i=0}^d w_i \sum_{\substack{u \in G \\ \text{dist}_G(v, u) \leq i}} L_{vu}^i x_u \end{aligned}$$

- ▶ Equivalentemente:

- ▶ **Solo contribuyen nodos con:** $\text{dist}_G(v, u) \leq i$

Localidad del Filtro

- ▶ La convolución en el nodo v depende únicamente de nodos u tales que:

$$\text{dist}_G(v, u) \leq d$$

- ▶ **Idea central:**

- ▶ Los filtros polinomiales son **locales**.
- ▶ El grado d controla el **alcance de la vecindad**.

Interpretación

- ▶ El filtro aplicado depende de la **fila correspondiente** de $p_w(\mathbf{L})$.
- ▶ **Observación clave:**
 - ▶ El “kernel” efectivo **varía según el nodo**.
 - ▶ Depende de la **posición del nodo en el grafo**.

Cálculo del Laplaciano del Grafo

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix} \quad \mathbf{D} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 5 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Laplaciano del grafo: $\mathbf{L} = \mathbf{D} - \mathbf{A}$

$$\mathbf{L} = \begin{bmatrix} 1 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 5 & -1 & -1 & -1 & -1 \\ 0 & 0 & -1 & 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Extensión de Convoluciones a Grafos

- ▶ La convolución con un filtro polinomial se define como:

$$\mathbf{x}' = p_{\mathbf{w}}(\mathbf{L})\mathbf{x}$$

donde

$$p_{\mathbf{w}}(\mathbf{L}) = w_0 \mathbf{I}_n + w_1 \mathbf{L} + w_2 \mathbf{L}^2 + \cdots + w_d \mathbf{L}^d = \sum_{i=0}^d w_i \mathbf{L}^i$$

- ▶ Sea la señal

$$\mathbf{x} = \begin{bmatrix} 1 \\ 7 \\ 3 \\ 6 \\ 7 \\ 1 \\ 6 \end{bmatrix}$$

Cálculo de Lx

$$Lx = \begin{bmatrix} 1 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 5 & -1 & -1 & -1 & -1 \\ 0 & 0 & -1 & 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 7 \\ 3 \\ 6 \\ 7 \\ 1 \\ 6 \end{bmatrix} = \begin{bmatrix} -6 \\ 10 \\ -12 \\ 3 \\ 4 \\ -2 \\ 3 \end{bmatrix}$$

Cálculo de L^2x

- ▶ Primero se aplica una *difusión local*: Lx
- ▶ Luego se vuelve a aplicar el mismo operador sobre ese resultado

$$L^2x = L(Lx) = \begin{bmatrix} 1 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 5 & -1 & -1 & -1 & -1 \\ 0 & 0 & -1 & 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -6 \\ 10 \\ -12 \\ 3 \\ 4 \\ -2 \\ 3 \end{bmatrix} = \begin{bmatrix} -16 \\ 38 \\ -78 \\ 15 \\ 16 \\ 10 \\ 15 \end{bmatrix}$$

Regla de actualización

- ▶ Dado:

$$x'_v = 0.8 x_v + 0.06 (Lx)_v + 0.002 (L^2x)_v$$

- ▶ Esto significa que cada nodo se actualiza como una combinación lineal de tres términos:
 - ▶ Su valor original
 - ▶ Información de vecinos (1 salto)
 - ▶ Información de vecinos de vecinos (2 saltos)

Nota: $L^2 = (D - A)^2 = D^2 - DA - AD + A^2$.

- ▶ Aparece: A^2 , caminos de longitud 2 (vecinos de vecinos),
- ▶ PERO también: D^2 , DA y AD , estos términos introducen pesos adicionales, restas, y combinaciones cruzadas

Actualización

$$x'_v = 0.8 x_v + 0.06 (Lx)_v + 0.002 (L^2 x)_v$$

$$x'_v = 0.8 \begin{bmatrix} 1 \\ 7 \\ 3 \\ 6 \\ 7 \\ 1 \\ 6 \end{bmatrix} + 0.06 \begin{bmatrix} -6 \\ 10 \\ -12 \\ 3 \\ 4 \\ -2 \\ 3 \end{bmatrix} + 0.002 \begin{bmatrix} -16 \\ 38 \\ -78 \\ 15 \\ 16 \\ 10 \\ 15 \end{bmatrix} = \begin{bmatrix} 0.408 \\ 6.276 \\ 1.524 \\ 5.01 \\ 5.872 \\ 0.7 \\ 5.01 \end{bmatrix}$$

Interpretación

- ▶ $0.8x_v$: mantiene la información original del nodo (memoria)
- ▶ $0.06(Lx)_v$: introduce diferencias locales (vecinos directos)
- ▶ $0.002(L^2x)_v$: introduce influencia de segundo orden (2 saltos)

ChebNet: Filtros Polinomiales Espectrales

- ▶ ChebNet propone una mejora sobre los filtros polinomiales en grafos.
- ▶ Se basa en la siguiente formulación:

$$p_w(L) = \sum_{i=1}^d w_i T_i(\tilde{L})$$

- ▶ donde:
 - ▶ $T_i(\cdot)$ es el **polinomio de Chebyshev de primer tipo** de grado i .
 - ▶ $\tilde{L}$ es el **Laplaciano normalizado**.

Definición del Laplaciano Normalizado

- ▶ El Laplaciano normalizado se define como:

$$\tilde{L} = \frac{2L}{\lambda_{\max}(L)} - I_n$$

- ▶ donde:
 - ▶ $\lambda_{\max}(L)$ es el **mayor autovalor** de L .
 - ▶ I_n es la **matriz identidad**.
- ▶ **Objetivo:** escalar el Laplaciano para controlar su comportamiento espectral.

Motivación: Estabilidad Numérica

- ▶ El Laplaciano L es **semidefinido positivo**:

$$\lambda_i(L) \geq 0$$

- ▶ Si $\lambda_{\text{máx}}(L) > 1$, entonces:
 - ▶ Las potencias de L pueden crecer rápidamente.
 - ▶ Esto genera **inestabilidad numérica**.
- ▶ **Problema clave**: explosión de valores en L^k .

Ventajas del Laplaciano Normalizado

- ▶ El Laplaciano normalizado $\tilde{L}$ tiene autovalores en:

$$[-1, 1]$$

- ▶ Esto implica:
 - ▶ **Control del crecimiento** de $\tilde{L}^k$.
 - ▶ **Mayor estabilidad** en los cálculos.
- ▶ **Idea central**: evitar que los coeficientes “exploten”.

Comparación: L vs $\tilde{L}$

- ▶ Usando L :
 - ▶ Se deben restringir los coeficientes de alto orden.
 - ▶ Riesgo de valores numéricos grandes.
- ▶ Usando $\tilde{L}$:
 - ▶ Se pueden usar coeficientes mayores.
 - ▶ Se mantiene una **escala controlada**.
- ▶ **Conclusión:** $\tilde{L}$ permite filtros más expresivos y estables.

Propiedades de los Polinomios de Chebyshev

- ▶ Los polinomios de Chebyshev poseen propiedades clave:
 - ▶ **Alta estabilidad numérica.**
 - ▶ Buen comportamiento en problemas de interpolación.
- ▶ Permiten:
 - ▶ Aproximaciones eficientes de funciones.
 - ▶ Construcción de filtros espectrales robustos.
- ▶ **Idea central:** base polinomial óptima para aproximación.

Cálculo de Embeddings en GNN

- ▶ Se construye una red neuronal sobre grafos apilando múltiples capas tipo ChebNet (o cualquier filtro polinomial) una tras otra, junto con no linealidades.
- ▶ En particular, si se tienen K capas diferentes de filtros polinomiales, donde la k -ésima capa posee sus propios pesos aprendibles $w^{(k)}$.
- ▶ Este **proceso es análogo a una CNN tradicional** con capas y no linealidades.
- ▶ **Idea central:** aplicar filtros polinomiales de forma iterativa para transformar representaciones.

Inicialización de Características

1. El proceso inicia con las características originales del grafo:

$$h^{(0)} = x$$

donde:

- ▶ x representa las **features iniciales de los nodos**.

Estructura por Capas

- ▶ Para una red con K capas, se itera:

$$k = 1, 2, \dots, K$$

- ▶ Cada capa k tiene sus propios **parámetros entrenables** $w^{(k)}$.
- ▶ **Idea central:** cada capa aprende un filtro polinomial distinto.

Definición del Filtro en la Capa k

2. Se define el filtro polinomial como:

$$p^{(k)} = p_{w^{(k)}}(L)$$

- ▶ Este filtro se obtiene evaluando el polinomio definido por los pesos del filtro $w^{(k)}$, en el Laplaciano L .
- ▶ **Interpretación:** el filtro captura información estructural del grafo.

Propagación Lineal

3. Se realiza la transformación lineal:

$$g^{(k)} = p^{(k)} h^{(k-1)}$$

Esto corresponde a:

- ▶ Un **producto matriz-vector**.
- ▶ Difusión de información entre nodos.

Idea central: agregación de información del vecindario.

Aplicación de No Linealidad

4. Se aplica una función de activación:

$$h^{(k)} = \sigma(g^{(k)})$$

donde σ es una función no lineal (por ejemplo, ReLU).

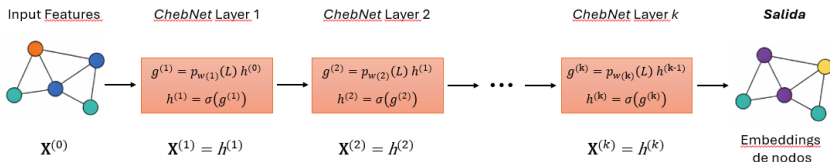
Idea central: introducir capacidad de modelado no lineal.

Compartición de Pesos

- ▶ Los pesos $w^{(k)}$ se comparten entre todos los nodos.
- ▶ Esto es equivalente al **weight sharing** en CNNs.
- ▶ Implicaciones:
 - ▶ Reducción del número de parámetros.
 - ▶ Invarianza respecto a la estructura del grafo.
- ▶ **Idea central:** mismo filtro aplicado globalmente.

Resumen del Proceso

- ▶ En cada capa se realizan tres pasos:
 1. Construcción del filtro polinomial $p^{(k)}$.
 2. Multiplicación con las embeddings previas.
 3. Aplicación de no linealidad.
- ▶ **Resultado:** embeddings progresivamente más ricos.



Configuración del Modelo

Caso simple (señal escalar por nodo)

- ▶ Cada nodo tiene una sola característica:

$$x \in \mathbb{R}^N$$

donde N es el número de nodos.

- ▶ **Idea central:** no existen matrices de características, únicamente un vector.

Filtro por Capa (Grado ℓ)

- ▶ Para un polinomio de grado ℓ :

$$x' = \sum_{i=0}^{\ell} w_i L^i x$$

donde: $w_i \in \mathbb{R}$, $i = 0, 1, \dots, \ell$

- ▶ L es fijo (Laplaciano del grafo).

Cada capa tiene $(\ell + 1)$ parámetros entrenables.

GNN con K Capas

- ▶ Para K capas:

$$\text{Capa } k : w_0^{(k)}, w_1^{(k)}, \dots, w_l^{(k)}, \quad k = 1, \dots, K$$

Cada capa aprende un filtro polinomial de grado ℓ .

Propagación en la Red

- ▶ La actualización en la capa k es:

$$h^{(k)} = \sigma \left(\sum_{i=0}^{\ell} w_i^{(k)} L^i h^{(k-1)} \right)$$

- ▶ **Interpretación:**

- ▶ $L^i h^{(k-1)}$ captura información a i saltos.
- ▶ Se combinan múltiples escalas de vecindad.

Total de Parámetros

- ▶ Número total de parámetros:

$$K \times (\ell + 1) = K(\ell + 1)$$

- ▶ Crecimiento lineal en capas y grado del polinomio.

Intuición del Modelo

- ▶ El modelo aprende:
 - ▶ **Peso del nodo propio** ($i = 0$).
 - ▶ **Vecinos a distintas distancias** ($i = 1, \dots, l$).
- ▶ Cada capa refina la representación:
 - ▶ Mezcla información local y global.
- ▶ **Limitación:**
 - ▶ No hay transformación de múltiples features.

Caso general (features vectoriales por nodo)

- ▶ Cada nodo tiene un vector de características:

$$X \in \mathbb{R}^{N \times F}$$

donde:

- ▶ N es el número de nodos
- ▶ F es el número de features por nodo

Filtro por Capa (Grado ℓ)

- ▶ Para un polinomio de grado ℓ :

$$X' = \sum_{i=0}^{\ell} L^i X \Theta_i, \quad \Theta_i \in \mathbb{R}^{F \times F'}$$

donde:

- ▶ F es el número de features de entrada por nodo
- ▶ F' es el número de features de salida por nodo

Cada capa combina difusión estructural y transformación de features.

GNN con K Capas

- ▶ Para K capas:

$$\text{Capa } k : \quad \Theta_0^{(k)}, \Theta_1^{(k)}, \dots, \Theta_\ell^{(k)}, \quad k = 1, \dots, K$$

- ▶ Cada capa aprende múltiples transformaciones asociadas a distintas escalas del grafo.

Propagación en la Red

- ▶ La actualización en la capa k es:

$$H^{(k)} = \sigma \left(\sum_{i=0}^{\ell} L^i H^{(k-1)} \Theta_i^{(k)} \right)$$

- ▶ **Interpretación:**

- ▶ $L^i H^{(k-1)}$: difusión hasta i saltos
- ▶ $\Theta_i^{(k)}$: transformación del espacio de características

Total de Parámetros

- ▶ Parámetros por capa: $(\ell + 1) F F'$
- ▶ Para K capas:

$$\sum_{k=1}^K (\ell + 1) F_{k-1} F_k$$

Intuición del Modelo

- ▶ El modelo aprende:
 - ▶ **Cómo mezclar vecinos a distintas distancias** ($i = 0, \dots, \ell$)
 - ▶ **Cómo transformar las características** ($F \rightarrow F'$)
- ▶ Cada nodo evoluciona como:

$$x_v \in \mathbb{R}^F \longrightarrow x'_v \in \mathbb{R}^{F'}$$

Combinación de estructura del grafo + aprendizaje de representaciones.

Comparación con el caso escalar

- ▶ Caso simple: solo mezcla de vecinos
- ▶ Caso general: mezcla vecinos + transformación de features

Aprendizaje de Parámetros en GNN

- ▶ Las operaciones en GNN (espectrales o espaciales) son **completamente diferenciables**.
- ▶ Esto permite entrenar el modelo de forma:
 - ▶ **End-to-end**, como en redes neuronales tradicionales.
- ▶ Requiere definir una función de pérdida $\mathcal{L}$ adecuada.

Clasificación de Nodos

- ▶ Se utilizan funciones de pérdida estándar, como la entropía cruzada:

$$\mathcal{L}(y_v, \hat{y}_v) = - \sum_c y_{vc} \log \hat{y}_{vc}$$

- ▶ donde:
 - ▶ $\hat{y}_{vc}$ es la probabilidad predicha para la clase c

Clasificación de Grafos

- ▶ Se agregan representaciones de nodos para obtener:
 - ▶ Un vector representativo del grafo completo.
- ▶ Este vector se utiliza en:
 - ▶ Clasificación de grafos
 - ▶ Otras tareas a nivel global
- ▶ **Idea central:** agregación de embeddings nodales (pooling).

Predicción de Enlaces

- ▶ Se consideran pares de nodos (v, u) :
 - ▶ Conectados y no conectados
- ▶ Probabilidad de conexión:

$$p_{vu} = \sigma(y_v^T y_u)$$

- ▶ Función de pérdida:

$$L(y_v, y_u, e_{vu}) = -e_{vu} \log(p_{vu}) - (1 - e_{vu}) \log(1 - p_{vu})$$

- ▶ donde:
 - ▶ $e_{vu} = 1$ si existe arista, 0 en otro caso
- ▶ **Idea central:** aprender similitud entre embeddings.

Redes Neuronales sobre Grafos (GNN)

- ▶ Las Redes Neuronales sobre Grafos (GNN) son modelos de aprendizaje profundo diseñados para datos estructurados como grafos.
 - ▶ Operan sobre:
 - ▶ Nodos
 - ▶ Aristas
 - ▶ Contexto global
 - ▶ Permiten aprender representaciones (embeddings) a partir de relaciones entre entidades.
-
- ▶ Una GNN es una transformación parametrizada y optimizable sobre los atributos de un grafo que preserva sus simetrías (invariancia ante permutaciones).
 - ▶ Respeta la estructura del grafo
 - ▶ No depende del orden de los nodos

Representación de un Grafo

- ▶ Un grafo se define como: $G = (V, E, U)$
 - ▶ V : conjunto de nodos
 - ▶ E : conjunto de aristas
 - ▶ U : atributos globales
- ▶ Cada nodo puede tener características: $x_i \in \mathbb{R}^d$

Mecanismo Fundamental: Message Passing

- ▶ Cada nodo actualiza su representación usando la información de sus vecinos.

$$h_i^{(k+1)} = \sigma \left(\text{AGG} \left(\{h_j^{(k)} : j \in \mathcal{N}(i)\} \right) \right)$$

- ▶ $h_i^{(k+1)}$: Representación (embedding) del nodo i en la capa $k+1$
- ▶ σ : función de activación
- ▶ AGG: función de agregación, combina la información de los vecinos
- ▶ $\{h_j^{(k)} : j \in \mathcal{N}(i)\}$: Conjunto de representaciones de todos los vecinos de i .

Proceso Iterativo

- ▶ Inicialización:

$$h_i^{(0)} = x_i$$

- ▶ En cada capa:
 - ▶ Se recibe información de vecinos
 - ▶ Se agrega la información
 - ▶ Se actualiza la representación
- ▶ Repetir permite capturar información de mayor alcance

Arquitectura Grafo-Entrada / Grafo-Salida

- ▶ Entrada: grafo con atributos en nodos y aristas
- ▶ Salida: mismo grafo con representaciones transformadas
- ▶ **La conectividad del grafo No cambia**
- ▶ **Solo cambian los embeddings.**

La GNN Más Simple

- ▶ Con la representación vectorial de grafos:
 - ▶ Es posible construir una GNN básica
 - ▶ En esta primera aproximación:
 - ▶ Se actualizan embeddings
 - ▶ **No se utiliza la conectividad del grafo**
-
- ▶ La GNN más simple aprende representaciones independientes para nodos, aristas y contexto global.

Estructura de la GNN Simple

- ▶ Se emplea un modelo diferenciable (típicamente un MLP):
 - ▶ Para nodos
 - ▶ Para aristas
 - ▶ Para el contexto global
- ▶ Cada componente se transforma de forma independiente:

$$h'_v = \text{MLP}_V(h_v), \quad h'_e = \text{MLP}_E(h_e), \quad h'_G = \text{MLP}_G(h_G)$$

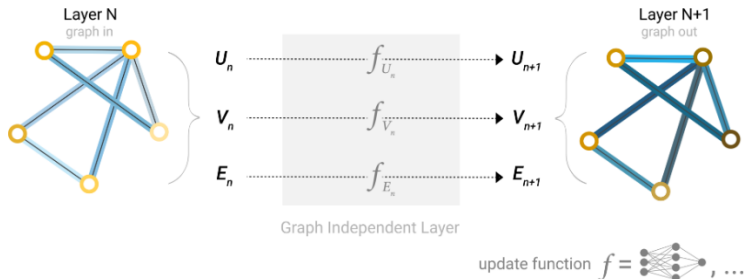


Figura tomada de: [Internet](#).

Interpretación

- ▶ Esta operación define una **capa GNN**:
 - ▶ Similar a capas en redes neuronales clásicas
- ▶ Características:
 - ▶ Transformaciones no lineales
 - ▶ Parámetros aprendibles
- ▶ Representación:

$$(V, E, U) \rightarrow (V', E', U')$$

Una capa GNN aplica transformaciones paralelas sobre todos los atributos del grafo.

Apilamiento de Capas

- ▶ Al igual que en redes profundas:
 - ▶ Las capas GNN pueden apilarse

- ▶ Evolución de embeddings:

$$h^{(0)} \rightarrow h^{(1)} \rightarrow h^{(2)} \rightarrow \dots$$

- ▶ Permite:
 - ▶ Mayor capacidad de representación

La profundidad permite aprender representaciones más complejas.

Propiedad de la Conectividad

- ▶ La estructura del grafo permanece constante durante el procesamiento.
- ▶ La GNN no modifica:
 - ▶ Número de nodos
 - ▶ Número de aristas
 - ▶ Lista de adyacencia
- ▶ Formalmente:

$$A' = A$$

Salida de la GNN

- ▶ La GNN transforma la información, no la topología.
- ▶ El grafo de salida:
 - ▶ Tiene la misma conectividad
 - ▶ Contiene nuevos embeddings

- ▶ Representación:

$$G' = (V', E', U')$$

- ▶ Donde:
 - ▶ V' , E' , U' son versiones transformadas

Limitación de la GNN Simple

- ▶ No se utiliza la información estructural:
 - ▶ No hay interacción entre nodos
- ▶ Consecuencia:
 - ▶ No captura dependencias del grafo
- ▶ Es un modelo base que no explota la conectividad, lo que motiva el **uso de message passing**.

Predicciones en GNN mediante Pooling

- ▶ Las GNN permiten realizar predicciones sobre nodos, aristas o el grafo completo mediante la **agregación de información estructurada**.
- ▶ **Caso simple:** Si existen **atributos en los nodos**, entonces:
 - ▶ Cada nodo i posee un embedding: $h_i \in \mathbb{R}^d$
 - ▶ La **predicción binaria** se obtiene como: $\hat{y}_i = \sigma(Wh_i + b)$
 - ▶ W y b son parámetros aprendidos
 - ▶ $\sigma(\cdot)$ es la función sigmoide

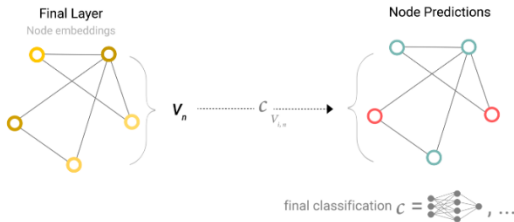


Figura tomada de: [Internet](#).

Limitaciones del Caso Simple

- ▶ No siempre se dispone de información en nodos.
- ▶ Ejemplo:
 - ▶ La información está en **aristas**
 - ▶ Se requiere predecir sobre **nod**os
- ▶ Problema: ¿Cómo transferir información desde las aristas a los nodos?
- ▶ Se utiliza el concepto de **pooling**.
- ▶ **Idea clave:**
 - ▶ Recolectar información desde múltiples elementos
 - ▶ Agregarla en una representación única

Proceso de Pooling

► 1. Recolección (Gathering)

- Se obtienen los embeddings relevantes
- Se concatenan en una matriz:

$$H = [h_1, h_2, \dots, h_k]^T$$

2. Agregación (Aggregation)

- Se combinan los embeddings
- Usualmente mediante suma:

$$h_{\text{pool}} = \sum_{j=1}^k h_j$$

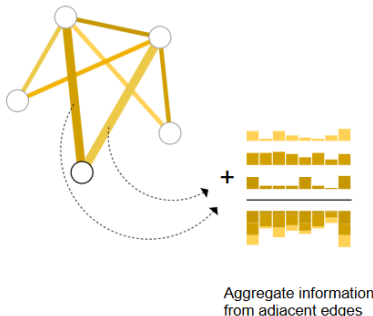


Figura tomada de: [Internet](#).

Operador de Pooling

- ▶ El pooling se denota como: $\rho(\cdot)$
- ▶ Transferencia de **Información de aristas** $\rightarrow$ **nodos**: $\rho_{E_n \rightarrow V_n}$
- ▶ Esto permite que los nodos reciban contexto estructural
- ▶ Caso: **solo existen características en aristas**. Entonces:
 - ▶ Aplicar pooling para obtener embeddings en nodos
 - ▶ Usar estos embeddings para predicción

Flujo del modelo:

1. Embeddings en aristas
 2. Pooling hacia nodos
 3. Clasificación en nodos
- ▶ El pooling permite **enrutar información** dentro del grafo para tareas de predicción.

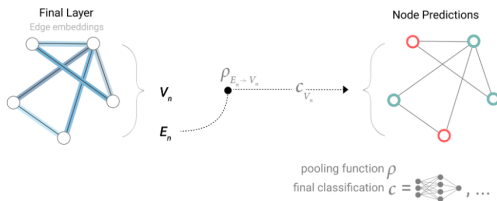


Figura tomada de: [Internet](#).

Redes Neuronales sobre Grafos (GNN)

- Si únicamente se dispone de características a nivel de nodos y se busca predecir información binaria a nivel de aristas.

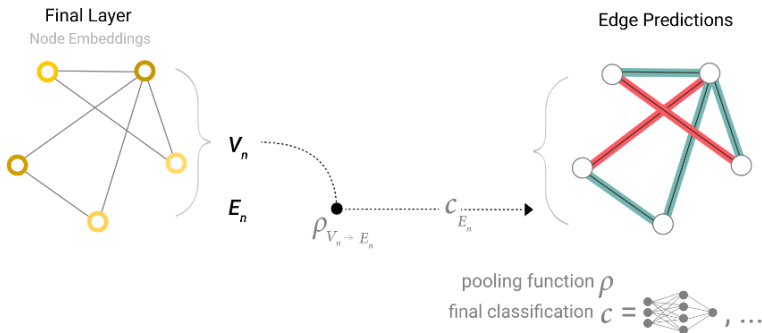


Figura tomada de: [Internet](#).

Redes Neuronales sobre Grafos (GNN)

- ▶ Si únicamente se dispone de características a nivel de nodos y se requiere predecir una propiedad global binaria, es necesario reunir toda la información disponible de los nodos y agregarla.
- ▶ Este proceso es similar a las capas de Global Average Pooling utilizadas en redes neuronales convolucionales (CNN).
- ▶ Este procedimiento también puede aplicarse a las aristas.

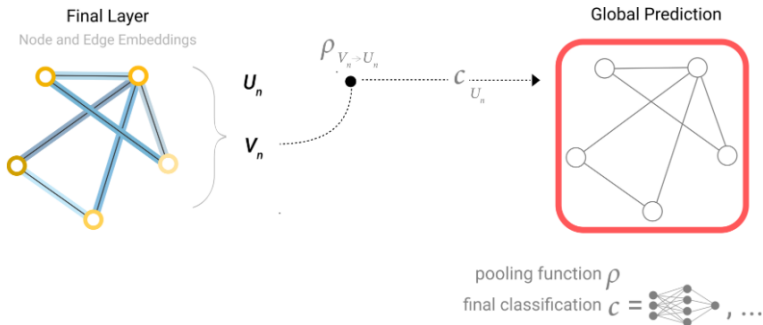


Figura tomada de: [Internet](#).

Redes Neuronales sobre Grafos (GNN)

- En los ejemplos considerados, el modelo de clasificación c puede reemplazarse fácilmente por cualquier modelo diferenciable, o adaptarse a clasificación multiclase mediante el uso de un modelo lineal generalizado.

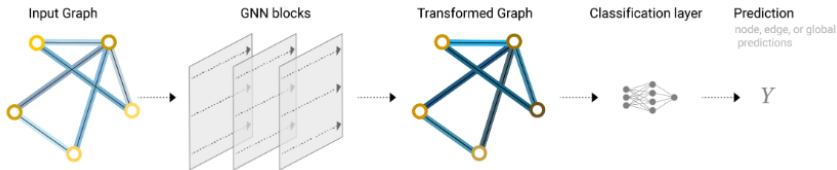


Figura tomada de: [Internet](#).

Pooling como Bloque Fundamental

- ▶ El pooling es un **bloque fundamental** en GNN.
- ▶ Permite construir modelos más complejos:
 - ▶ Integrando múltiples tipos de atributos
 - ▶ Definiendo rutas de información
- ▶ **Principio general:**
 - ▶ Para cada nuevo atributo del grafo, se debe definir **cómo fluye la información** hacia otros atributos

Flujo de Información en GNN

- ▶ En un grafo pueden existir:
 - ▶ Atributos en nodos
 - ▶ Atributos en aristas
 - ▶ Atributos globales
- ▶ El modelo debe especificar:
 - ▶ **Qué información se transfiere**
 - ▶ **Desde dónde y hacia dónde**
- ▶ Ejemplo: Aristas → Nodos (mediante pooling)

Limitación del Modelo GNN Simple

- ▶ En la formulación básica **no se utiliza la conectividad del grafo** dentro de la capa GNN
- ▶ En consecuencia:
 - ▶ Cada nodo se procesa de forma **independiente**
 - ▶ Cada arista se procesa de forma **independiente**
 - ▶ El contexto global también se trata de forma independiente
- ▶ La conectividad del grafo solo se emplea durante el **pooling**
- ▶ Específicamente:
 - ▶ Para determinar **qué elementos se agregan**
 - ▶ En el proceso de transferencia de información
- ▶ **Interpretación:**
 - ▶ La estructura del grafo influye en la predicción, pero no en la transformación interna inicial

Message Passing en GNN

- ▶ Es posible mejorar las predicciones en GNN mediante:
 - ▶ El uso de **pooling dentro de la capa GNN**
 - ▶ La incorporación de la **conectividad del grafo**
- ▶ **Idea clave:** Los embeddings deben ser **sensibles a la estructura del grafo**
- ▶ Esto se logra mediante el mecanismo de: **Message Passing**
- ▶ El message passing permite:
 - ▶ El **intercambio de información** entre nodos vecinos
 - ▶ La **influencia mutua** entre sus representaciones
- ▶ Resultado: Los embeddings se actualizan considerando el **contexto estructural local**
- ▶ **Conclusión:** El modelo deja de ser independiente por nodo y pasa a ser **dependiente del grafo**

Etapas del Message Passing

1. Recolección (Gathering)

- ▶ Para cada nodo i , se obtienen los embeddings de sus vecinos:

$$\{h_j : j \in \mathcal{N}(i)\}$$

2. Agregación (Aggregation)

- ▶ Se combinan los mensajes:

$$m_i = \sum_{j \in \mathcal{N}(i)} h_j$$

3. Actualización (Update)

- ▶ Se actualiza el embedding del nodo:

$$h'_i = \phi(h_i, m_i)$$

- ▶ La función de actualización suele ser:

- ▶ Una **red neuronal entrenable**

- ▶ Forma general:

$$h_i^{(k+1)} = \sigma(W \cdot m_i + b)$$

- ▶ Donde:

- ▶ m_i es la información agregada de los vecinos
 - ▶ W, b son parámetros aprendidos
 - ▶ σ es una función de activación

- ▶ **Interpretación:**

- ▶ Cada nodo aprende a integrar información local del grafo

Redes Neuronales sobre Grafos (GNN)

Ejemplo.

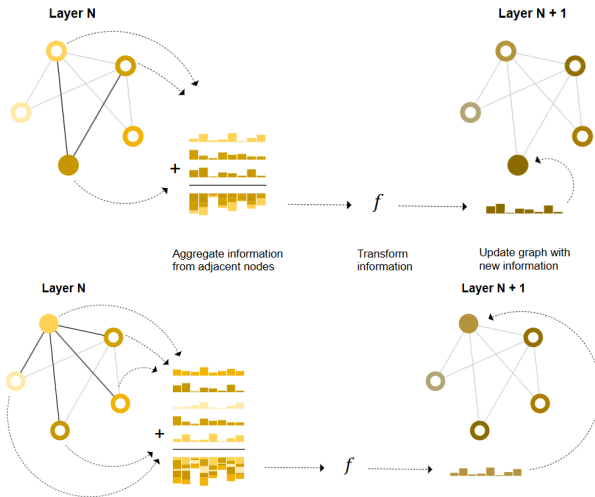


Figura tomada de: [Internet](#).

Generalización del Message Passing

- ▶ El message passing no se limita a nodos:
 - ▶ Puede aplicarse entre nodos
 - ▶ Puede aplicarse entre aristas
- ▶ Similar al pooling, define **cómo fluye la información** dentro del grafo
- ▶ **Flexibilidad:** Permite modelar diferentes tipos de relaciones estructurales

Importancia del Message Passing

- ▶ Clave para **explotar la conectividad del grafo**
- ▶ Permite:
 - ▶ Construir modelos GNN más expresivos
 - ▶ Capturar dependencias estructurales complejas
- ▶ **Conclusión:** El message passing incrementa la **capacidad representacional** del modelo

Capa de Message Passing en GNN

- ▶ La secuencia de operaciones de message passing define:
 - ▶ La **forma más simple de una capa GNN**
- ▶ Componentes:
 - ▶ Recolección de vecinos
 - ▶ Agregación de información
 - ▶ Actualización de embeddings
- ▶ **Resultado:**
 - ▶ Embeddings dependientes de la estructura del grafo

Relación con la Convolución

- ▶ El message passing es análogo a la **convolución en redes neuronales**
- ▶ Idea común:
 - ▶ Agregar información de los **vecinos**
 - ▶ Actualizar la representación del elemento central
- ▶ Comparación:
 - ▶ En imágenes: el elemento es un **píxel**
 - ▶ En grafos: el elemento es un **nodo**

Diferencia Clave con CNN

- ▶ En imágenes:
 - ▶ Cada píxel tiene un **número fijo de vecinos**
- ▶ En grafos:
 - ▶ Cada nodo tiene un **número variable de vecinos**
- ▶ Implicación:
 - ▶ Se requieren funciones de agregación invariantes
 - ▶ Ejemplo: $AGG = \sum, \text{mean}, \text{max}$

Propagación de Información

- ▶ Al apilar múltiples capas GNN:
 - ▶ Se incrementa el **alcance de la información**
- ▶ Después de k capas:
 - ▶ Un nodo incorpora información de vecinos a distancia k
- ▶ Ejemplo:
 - ▶ 1 capa $\rightarrow$ vecinos directos
 - ▶ 2 capas $\rightarrow$ vecinos de vecinos
 - ▶ 3 capas $\rightarrow$ nodos a tres saltos
- ▶ La actualización en múltiples capas se expresa como:

$$h_i^{(k+1)} = \sigma \left(W \cdot \sum_{j \in \mathcal{N}(i)} h_j^{(k)} \right)$$

donde:

- ▶ k indica la capa
- ▶ $\mathcal{N}(i)$ es el vecindario del nodo i
- ▶ **Interpretación:**
 - ▶ La información se propaga progresivamente en el grafo.

Actualización de la Arquitectura

- ▶ La arquitectura del modelo se extiende incorporando múltiples capas de message passing
- ▶ Cada capa:
 - ▶ Amplía el contexto de cada nodo
 - ▶ Enriquece los embeddings
- ▶ **Conclusión:** Las GNN profundas capturan información global a partir de interacciones locales

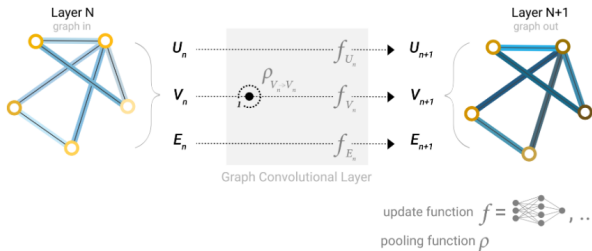


Figura tomada de: [Internet](#).

Aprendizaje de Representaciones en Aristas

- ▶ En muchos casos, el dataset:
 - ▶ **No contiene todos los tipos de información**
 - ▶ (nodos, aristas, contexto global)
- ▶ Problema:
 - ▶ Se desea predecir sobre **nodos**
 - ▶ Pero solo existen atributos en **aristas**
- ▶ Solución previa:
 - ▶ Uso de **pooling** en la etapa final

Limitación del Pooling Final

- ▶ El pooling aplicado solo al final:
 - ▶ No permite interacción durante el aprendizaje
- ▶ Consecuencia: Los embeddings no capturan relaciones profundas
- ▶ Se requiere compartir información **dentro de la capa GNN**

Message Passing entre Aristas y Nodos

- ▶ El message passing permite:
 - ▶ Transferir información entre **aristas y nodos**
- ▶ Estrategia:
 - ▶ Recolectar información de aristas vecinas
 - ▶ Agregarla mediante pooling
 - ▶ Aplicar una función de actualización
- ▶ **Resultado:** Embeddings enriquecidos en nodos y aristas

Agregación de Información en Aristas

- ▶ Para cada nodo i , se consideran sus aristas incidentes:

$$\mathcal{E}(i)$$

- ▶ Se agregan los embeddings de aristas:

$$m_i^{(e)} = \sum_{e \in \mathcal{E}(i)} h_e$$

- ▶ Luego se actualiza el nodo:

$$h'_i = \phi(h_i, m_i^{(e)})$$

- ▶ El nodo incorpora información estructural de sus conexiones

Redes Neuronales sobre Grafos (GNN)

Ejemplo:

- ▶ El primer paso “prepara” un mensaje compuesto por información de una arista y de los nodos conectados a ella, y
- ▶ luego “transmite” el mensaje al nodo.

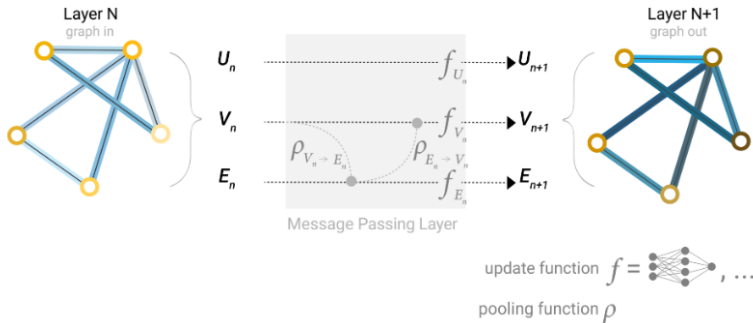


Figura tomada de: [Internet](#).

Problema de Compatibilidad

- ▶ Los embeddings pueden diferir en: Dimensión y/o Forma
- ▶ Ejemplo: $h_i \in \mathbb{R}^{d_v}$ (nodos); $h_e \in \mathbb{R}^{d_e}$ (aristas)
- ▶ Problema: **No es trivial combinarlos directamente**

Estrategia 1: Mapeo Lineal

- ▶ Se aprende una transformación lineal: $\tilde{h}_e = W_e h_e$, donde:
 $W_e \in \mathbb{R}^{d_v \times d_e}$
- ▶ Permite proyectar aristas al espacio de nodos
- ▶ **Ventaja:** Compatibilidad dimensional

Estrategia 2: Concatenación

- ▶ Concatenar embeddings: $z_i = [h_i \parallel m_i^{(e)}]$
- ▶ Luego aplicar: $h'_i = \phi(z_i)$
- ▶ **Ventaja:** Preserva toda la información original
- ▶ **Desventaja:** Aumenta la dimensionalidad

Diseño de Actualizaciones en GNN

- ▶ En el diseño de una GNN, una decisión clave es:
 - ▶ **Qué atributos del grafo se actualizan**
 - ▶ **En qué orden se actualizan**
- ▶ Atributos posibles: Nodos; Aristas; Contexto global
- ▶ **Impacto:** Determina la capacidad del modelo para capturar relaciones

Orden de Actualización

- ▶ Existen múltiples estrategias:
 - ▶ Actualizar primero **nodos** y luego **aristas**
 - ▶ Actualizar primero **aristas** y luego **nodos**
- ▶ Cada elección implica:
 - ▶ Diferentes flujos de información
 - ▶ Diferentes representaciones aprendidas
- ▶ **Conclusión:** No existe una única estrategia óptima
- ▶ El diseño de actualizaciones en GNN es:
 - ▶ Un **problema abierto de investigación**

Estrategia: Actualización Tipo “Weave”

- ▶ Combina múltiples flujos de información entre nodos y aristas
- ▶ Se generan varias representaciones intermedias:
 - ▶ **Nodo a Nodo ($N \rightarrow N$):**

$$h'_i = W_n h_i$$

- ▶ **Arista a Arista ($E \rightarrow E$):**

$$h'_e = W_e h_e$$

- ▶ **Nodo a Arista ($N \rightarrow E$):**

- ▶ Se construyen representaciones de aristas a partir de nodos

- ▶ **Arista a Nodo ($E \rightarrow N$):**

- ▶ Se agregan aristas incidentes al nodo

Redes Neuronales sobre Grafos (GNN)

- Algunas de las formas en que se pueden combinar las representaciones de aristas y nodos en una capa de una GNN.

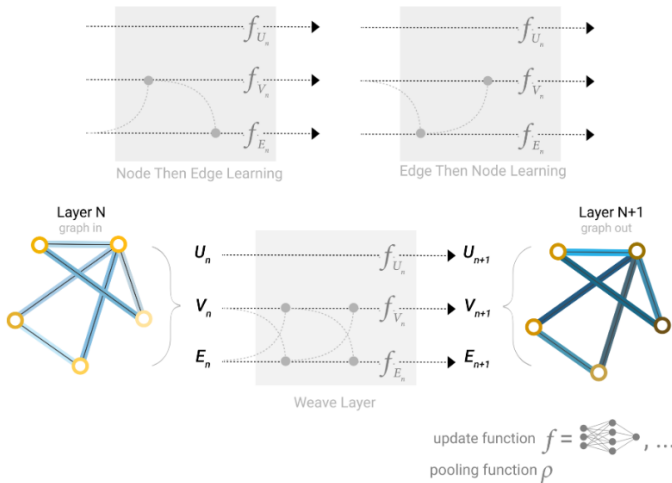


Figura tomada de: [Internet](#).

Combinación de Representaciones

- ▶ Las representaciones intermedias se combinan:
 - ▶ Para generar nuevos embeddings de nodos y aristas
- ▶ Forma general:

$$h_i^{(new)} = \phi(h_i^{NN}, h_i^{EN})$$

$$h_e^{(new)} = \phi(h_e^{EE}, h_e^{NE})$$

- ▶ **Ventaja:**
 - ▶ Integración rica de múltiples fuentes de información

Representaciones Globales en GNN

- ▶ Limitación de las GNN previas:
 - ▶ Los nodos lejanos tienen **dificultad para intercambiar información**
- ▶ El message passing es inherentemente: **Local**
- ▶ Consecuencia:
 - ▶ La información entre nodos distantes requiere muchas capas
- ▶ Problemas asociados:
 - ▶ Mayor costo computacional
 - ▶ Posible degradación del aprendizaje

Solución 1: Conectividad Total

- ▶ Conectar todos los nodos entre sí
- ▶ Conocido como: “Virtual Edges”
- ▶ **Desventaja:** Altamente costoso en grafos grandes
- ▶ Aplicación: Grafos pequeños (ej. moléculas)

Solución 2: Representación Global

- ▶ Alternativa eficiente: Introducir una **representación global**
- ▶ Denotada como:

$$u \in \mathbb{R}^{d_u}$$

- ▶ También conocida como:
 - ▶ **Master Node**
 - ▶ **Context Vector**

Rol de la Representación Global

- ▶ El vector global se conecta a:
 - ▶ Todos los nodos
 - ▶ Todas las aristas
- ▶ Función: Actuar como **punto de comunicación**
- ▶ Permite la transferencia eficiente de información global

Actualización del Contexto Global

- ▶ El contexto global se actualiza como:

$$u' = \phi \left(u, \sum_i h_i, \sum_e h_e \right)$$

donde:

- ▶ h_i son embeddings de nodos
- ▶ h_e son embeddings de aristas
- ▶ **Interpretación:** El vector global resume información del grafo completo

Redes Neuronales sobre Grafos (GNN)

- Esquema de una arquitectura de Graph Nets que aprovecha representaciones globales.

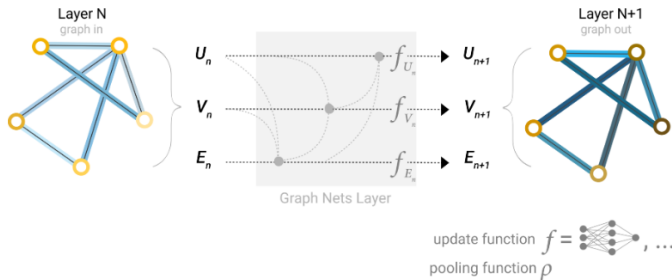


Figura tomada de: [Internet](#).

Condicionamiento de la Información

- ▶ En esta formulación: Todos los atributos del grafo poseen **representaciones aprendidas**
- ▶ El embedding de un nodo puede depender de:
 - ▶ Nodos vecinos
 - ▶ Aristas conectadas
 - ▶ Información global
- ▶ La representación se construye **condicionando** múltiples fuentes de información:
 - ▶ Concatenación
 - ▶ Mapeo a un espacio común
 - ▶ Modulación por características
- ▶ Esto permite capturar relaciones complejas en el grafo

Concatenación de Representaciones

- ▶ Una estrategia directa es:
 - ▶ Concatenar todas las fuentes de información
- ▶ Para un nodo i :

$$z_i = [h_i \parallel m_i^{(n)} \parallel m_i^{(e)} \parallel u]$$

- ▶ Donde:
 - ▶ $m_i^{(n)}$: información de nodos vecinos
 - ▶ $m_i^{(e)}$: información de aristas
 - ▶ u : contexto global
- ▶ **Ventaja:**
 - ▶ Preserva toda la información disponible

Mapeo a un Espacio Común

- ▶ Mapear todas las representaciones a un mismo espacio, mediante transformaciones lineales:

$$\tilde{h}_i = W_n h_i, \quad \tilde{m}_i^{(e)} = W_e m_i^{(e)}, \quad \tilde{u} = W_u u$$

- ▶ Luego combinar:

$$h'_i = \tilde{h}_i + \tilde{m}_i^{(n)} + \tilde{m}_i^{(e)} + \tilde{u}$$

- ▶ **Ventaja:** Control de dimensionalidad

Modulación por Características

- ▶ Forma general:

$$h'_i = \gamma(u) \odot h_i + \beta(u)$$

- ▶ Donde:
 - ▶ $\gamma(u)$ y $\beta(u)$ dependen del contexto global
 - ▶ $\odot$ es el producto elemento a elemento
- ▶ **Interpretación:** Mecanismo similar a **atención sobre características**

Redes Neuronales sobre Grafos (GNN)

- ▶ Esquema para condicionar la información de un nodo a partir de tres embeddings adicionales (nodos adyacentes, aristas adyacentes y representación global).
- ▶ Este paso corresponde a las operaciones sobre nodos en la capa de Graph Nets.

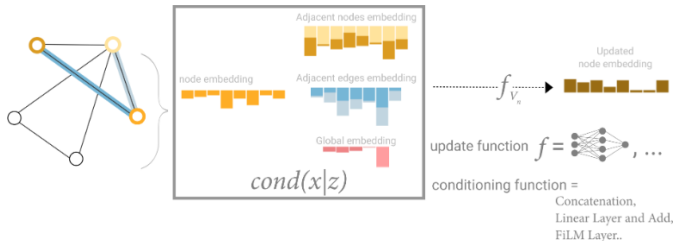


Figura tomada de: [Internet](#).

Graph Neural Network en Python

Se introduce un ejemplo básico de construcción de una Graph Neural Network (GNN) aplicado a un **conjunto de datos de una red social**^a.

En este contexto:

- ▶ Cada nodo representa una persona.
- ▶ Cada arista representa una relación de amistad entre dos personas.
- ▶ Cada nodo posee características asociadas.

^aMaterial tomado de: [A Gentle Introduction to Graph Neural Networks in Python](#).

Representación del Grafo

- ▶ Los nodos contienen atributos como:
 - ▶ Edad
 - ▶ Intereses
 - ▶ Otras características personales
- ▶ Las aristas modelan las conexiones sociales (amistades).
- ▶ Esta estructura permite representar tanto información individual como relacional.

Tarea de Predicción

- ▶ El objetivo de la GNN es realizar una **tarea de clasificación binaria**:
 - ▶ Clasificar a cada persona como: Popular o No popular
 - ▶ El criterio se basa en:
 - ▶ Tener más de dos amigos
 - ▶ Tener dos o menos amigos

Información Considerada

- ▶ Las características individuales de cada persona:
 - ▶ Intereses
 - ▶ Edad, entre otros
- ▶ Las conexiones con otras personas:
 - ▶ Estructura del grafo
 - ▶ Relaciones de vecindad

Ventajas de las GNN

- ▶ Las GNN no solo consideran las características del nodo objetivo.
- ▶ También incorporan información relacional del entorno.
- ▶ Permiten capturar dependencias estructurales del grafo.

Esto las diferencia de:

- ▶ Modelos clásicos de clasificación
- ▶ Modelos de regresión tradicionales

Mecanismo Fundamental: Message Passing

- ▶ La idea base en GNN es:

$$h_i^{(k+1)} = \sigma \left(\text{AGG} \left(\{h_j^{(k)} : j \in \mathcal{N}(i)\} \right) \right)$$

donde:

- ▶ $h_i^{(k+1)}$: Representación (embedding) del nodo i en la capa $k + 1$
- ▶ σ : función de activación
- ▶ AGG: función de agregación, combina la información de los vecinos
- ▶ $\{h_j^{(k)} : j \in \mathcal{N}(i)\}$: Conjunto de representaciones de todos los vecinos de i ($\mathcal{N}(i)$).

Cada nodo:

- ▶ recoge información de sus vecinos
- ▶ la combina
- ▶ aplica una transformación no lineal

Forma Básica de Agregación (Lineal Simple)

- ▶ Una implementación simple de AGG es:

$$h_i^{(k+1)} = \sigma \left(\sum_{j \in \mathcal{N}(i)} h_j^{(k)} W^{(k)} \right)$$

- ▶ **Problema:**

- ▶ Nodos con muchos vecinos generan valores más grandes
- ▶ No hay normalización
- ▶ La suma directa produce inestabilidad.

Uso de la Matriz de Adyacencia

- ▶ La agregación puede escribirse como:

$$H^{(k+1)} = \sigma \left(A H^{(k)} W^{(k)} \right)$$

- ▶ donde:

$$A \in \mathbb{R}^{N \times N}, \quad H^{(k)} \in \mathbb{R}^{N \times F}$$

$$W \in \mathbb{R}^{F \times F'}$$

- ▶ F : número de features de entrada
- ▶ F' : número de features de salida (dimensión del embedding en la siguiente capa)
- ▶ Interpretación:

$$A H^{(k)} \Rightarrow \text{suma de vecinos}$$

Incluir Información Propia

- ▶ Se agregan auto-conexiones:

$$\tilde{A} = A + I$$

- ▶ Nueva agregación:

$$\tilde{A}H^{(k)}$$

- ▶ Ahora incluye:

- ▶ vecinos
- ▶ el propio nodo

- ▶ **Cada nodo conserva su propia información.**

Problema de Escalamiento

- ▶ Problema: Nodos con muchos vecinos producen valores grandes
- ▶ Consecuencia: Activaciones inestables
- ▶ Solución: **Normalización por grado**

Matriz de Grados

- ▶ Se define:

$$\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$$

- ▶ $\tilde{D}$ es una matriz diagonal.
- ▶ Opción de normalización simple:

$$\tilde{D}^{-1} \tilde{A}$$

- ▶ Problema: **No es simétrica**

Operador Normalizado

- ▶ Se utiliza:

$$\hat{A} = \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}}$$

- ▶ **Ventajas:**

- ▶ operador simétrico
- ▶ mayor estabilidad numérica
- ▶ conexión con el Laplaciano normalizado

- ▶ **Idea central:** normalización balanceada entre nodos.

Interpretación

- ▶ Cada mensaje se escala como:

$$\frac{1}{\sqrt{d_i d_j}}$$

donde: d_i, d_j : grados de los nodos

- ▶ La normalización evita dominancia de nodos muy conectados.

Transformación de Features

- ▶ Después de agregar:

$$\hat{A}H^{(k)}$$

- ▶ Se aplica transformación:

$$(\hat{A}H^{(k)})W^{(k)}$$

donde:

$$W^{(k)} \in \mathbb{R}^{F \times F'}$$

- ▶ **Idea central:** cambio de espacio de características.

Modelo Final (Capa de GCN)

- ▶ La actualización final es:

$$H^{(k+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(k)} W^{(k)} \right)$$

- ▶ **Idea central:**
 - ▶ agregación normalizada
 - ▶ transformación lineal
 - ▶ no linealidad
- ▶ **Resultado:** embeddings estables y expresivos.

Pasos de la Convolución en Grafos

1. Se agregan auto-conexiones:

$$\tilde{A} = A + I$$

2. Se calcula la matriz de grados:

$$\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$$

3. Se normaliza la matriz de adyacencia:

$$\tilde{A}_{norm} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$$

4. Se aplica la transformación lineal:

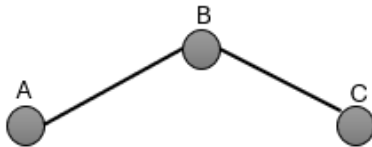
$$H^{(l)} W^{(l)}$$

5. Se agregan las características de los vecinos y se aplica la activación:

$$H^{(k+1)} = \sigma \left(\tilde{A}_{norm} H^{(k)} W^{(k)} \right)$$

Ejemplo numérico simple:

Sea el grafo simple con 3 nodos:



- **Matriz de adyacencia** A . Ordenando los nodos como (A, B, C) :

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

► 1. Agregar autoconexiones

$$\tilde{A} = A + I = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

► 2. Matriz de grados $\tilde{D}$

Sumando por filas:

- Nodo A: 2
- Nodo B: 3
- Nodo C: 2

$$\tilde{D} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 2 \end{bmatrix}$$

► 3. Normalización $\tilde{D}^{-1/2}$

$$\tilde{D}^{-1/2} = \begin{bmatrix} \frac{1}{\sqrt{2}} & 0 & 0 \\ 0 & \frac{1}{\sqrt{3}} & 0 \\ 0 & 0 & \frac{1}{\sqrt{2}} \end{bmatrix}$$

► Matriz normalizada

$$\tilde{A}_{\text{norm}} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$$

Resultado:

$$\tilde{A}_{\text{norm}} = \begin{bmatrix} \frac{1}{2} & \frac{1}{\sqrt{6}} & 0 \\ \frac{1}{\sqrt{6}} & \frac{1}{3} & \frac{1}{\sqrt{6}} \\ 0 & \frac{1}{\sqrt{6}} & \frac{1}{2} \end{bmatrix}$$

Ejemplo introductorio

Se considera un conjunto de datos asociado a una **plataforma de redes sociales**, donde:

- ▶ Cada **nodo** representa una persona
- ▶ Cada **arista** representa una relación de amistad

Además, cada nodo cuenta con **características asociadas**:

- ▶ Edad
- ▶ Intereses
- ▶ Otros atributos

Tarea objetivo: clasificación binaria

- ▶ Personas **populares** o **no populares**

La clasificación se basa en:

- ▶ Las **características propias** del nodo
- ▶ Las **conexiones** con otros nodos

Conjunto de Datos: Mini Red Social

Se define un pequeño conjunto de datos en forma de **grafo**, donde:

- ▶ Los **nodos** representan personas
- ▶ Las **aristas** representan relaciones de amistad

Representación de Conexiones: `edge_index`

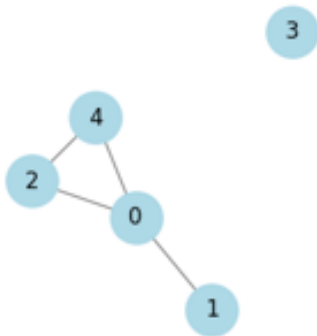
- ▶ La matriz `edge_index` describe las conexiones entre los usuarios
- ▶ Existen **5 usuarios**, identificados de 0 a 4

$$\begin{bmatrix} 0, 1, 0, 2, 0, 4, 2, 4, \\ 1, 0, 2, 0, 4, 0, 4, 2, \end{bmatrix},$$

- ▶ Cada columna de la matriz representa una **conexión dirigida**:
 - ▶ $0 \rightarrow 1$: conexión del usuario 0 al 1
 - ▶ $1 \rightarrow 0$: conexión inversa

Observación:

- ▶ El usuario 3 no presenta conexiones
- ▶ Se encuentra **aislado** dentro del grafo



Ver código python.

- ▶ A partir de `edge_index`, se construye la matriz de adyacencia:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \end{bmatrix}$$

- ▶ Se definen las **Características iniciales**:

$$H^{(0)} = \begin{bmatrix} 25 & 1 \\ 30 & 0 \\ 22 & 1 \\ 35 & 0 \\ 27 & 1 \end{bmatrix} \begin{array}{l} \text{Person 0 (25 years old, likes sports)} \\ \text{Person 0 (25 years old, likes sports)} \\ \text{Person 0 (25 years old, likes sports)} \\ \text{Person 0 (25 years old, likes sports)} \\ \text{Person 0 (25 years old, likes sports)} \end{array}$$

Construcción de un Modelo de Red Neuronal

► Definición de etiquetas

- Se definen las etiquetas del conjunto de datos de usuarios, determinando si una persona es **popular** o **no popular**.
- La clasificación se basa en el número de **conexiones (amigos)** dentro de la red.

► Proceso:

- Se calcula el número de conexiones de cada nodo
- A partir de la **matriz de adyacencia**
- Esto define el valor real (*ground truth*)

► Regla de clasificación: $$\begin{cases} \geq 2 \text{ amigos} \rightarrow \text{popular (1)} \\ < 2 \text{ amigos} \rightarrow \text{no popular (0)} \end{cases}$$

► Etiquetas:

```
num_friends = tensor([3, 1, 2, 0, 3])  
labels = tensor([1, 0, 1, 0, 1])
```


Máscara de entrenamiento

- ▶ Se define una **máscara** para dividir los datos del grafo:
 - ▶ Las **tres primeras personas** se usan para entrenamiento
 - ▶ Las **dos restantes** se reservan para inferencia/evaluación

`train_mask: tensor([ True, True, True, False, False])`

Estructura de datos

- ▶ Toda la información se encapsula en un objeto `Data`
- ▶ Este objeto contiene:
 - ▶ Estructura del grafo (`edge_index`)
 - ▶ Características de los nodos
 - ▶ Etiquetas
 - ▶ Máscara de entrenamiento

`data: Data(x=[5, 2], edge_index=[2, 8], y=[5], train_mask=[5])`

Arquitectura del modelo

- ▶ Se define la arquitectura de la **Red Neuronal en Grafos (GNN)** y se instancia el modelo.
- ▶ En **PyTorch Geometric**, las GNN se construyen mediante capas de convolución sobre grafos, como GCNConv.
- ▶ **Idea clave:**
 - ▶ Las capas agregan información de los **nodos vecinos**
 - ▶ Permiten aprender representaciones que capturan:
 - ▶ Características individuales
 - ▶ Estructura del grafo

Fórmula Central de GCNConv

$$H'^{(k+1)} = \hat{A} H^{(k)} W^{(k)} + b^{(k)}$$

donde:

- ▶ $H^{(k)} \in \mathbb{R}^{N \times F_{\text{in}}}$, matriz de features de los nodos en la capa k
- ▶ N , número de nodos
- ▶ F_{in} , número de features de entrada
- ▶ $W^{(k)} \in \mathbb{R}^{F_{\text{in}} \times F_{\text{out}}}$, matriz de pesos entrenables
- ▶ $b^{(k)} \in \mathbb{R}^{F_{\text{out}}}$, sesgos (bias) entrenable
- ▶ $\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$, matriz de adyacencia normalizada

Finalmente:

$$H^{(k+1)} = \sigma \left(H'^{(k+1)} \right)$$

donde σ es una función que introduce no linealidad.

► Arquitectura utilizada:

$\text{GCNConv}(2 \rightarrow 4) \rightarrow \text{ReLU} \rightarrow \text{GCNConv}(4 \rightarrow 2)$

► Interpretación:

- Primera capa: transforma de 2 a 4 características
- ReLU: introduce no linealidad
- Capa de salida: proyecta a 2 clases

► Cada capa:

- Agrega información de vecinos
- Aprende representaciones del nodo considerando el grafo

```
=====
Layer (type:depth-idx)                   Param #
=====
GNN                                         --
├─GCNConv: 1-1                             4
│   └─SumAggregation: 2-1                 --
│       └─Linear: 2-2                     8
├─GCNConv: 1-2                             2
│   └─SumAggregation: 2-3                 --
│       └─Linear: 2-4                     8
=====
Total params: 22
Trainable params: 22
Non-trainable params: 0
=====
```

Entrenamiento

- ▶ El proceso de entrenamiento de una GNN es similar al de otros modelos en PyTorch.
- ▶ **Definición del optimizador:**
`optimizer = torch.optim.Adam(model.parameters(), lr=0.01)`
- ▶ **Bucle de entrenamiento:**
`for epoch in range(400):
 model.train()
 optimizer.zero_grad()
 out = model(data)
 loss = F.cross_entropy(out[data.train_mask],
 data.y[data.train_mask])
 loss.backward()
 optimizer.step()`

Notas:

- ▶ Se utiliza `train_mask` para entrenar solo con los nodos designados
- ▶ La función de pérdida es `cross_entropy`, usada en clasificación
- ▶ `optimizer.step()` actualiza los pesos del modelo

Inferencia en una Red Neuronal en Grafos en Python

- ▶ Una vez entrenada la GNN, la inferencia es directa.
- ▶ **Proceso:**
 - ▶ Se utiliza el **conjunto completo de datos** para calcular predicciones
 - ▶ Se incluyen los usuarios que no fueron considerados durante el entrenamiento
 - ▶ Se obtiene la clase predicha de cada nodo usando `argmax`

▶ Ejemplo de código en PyTorch:

```
model.eval()  
with torch.no_grad():  
    pred = model(data)  
    predicted_class = pred.argmax(dim=1)
```

▶ Nota:

- ▶ El uso de `argmax` selecciona la clase con mayor
- ▶ probabilidad, típico en clasificadores binarios (como la regresión logística).

Gracias por su atención.

